Passenger Forecasting Report (2026–2030)

1. Introduction

The objective of this analysis is to forecast passenger demand at the Port Authority Bus Terminal for the period 2026–2030. Accurate forecasting is important for planning temporary staging facilities, optimizing resource allocation, and supporting operational decision-making. This study applies time series forecasting techniques to understand future passenger trends and support data-driven planning.

2. Tools Used

The following tools and libraries were used for the analysis:

- Python (main programming language)
- Pandas (data preprocessing and aggregation)
- Statsmodels (Exponential Smoothing model)
- pmdarima (Auto ARIMA model selection)
- Matplotlib (data visualization and plotting)

3. Methods / Techniques Used

Two time series forecasting models were implemented: Exponential Smoothing and ARIMA, and their performance was evaluated using statistical error metrics.

3.1 Exponential Smoothing (Holt's Linear Trend Model)

This model captures the trend in time series data using an additive trend component. It is suitable for datasets that show a steady increase or decrease over time without strong seasonal patterns.

3.2 ARIMA (Auto ARIMA)

The ARIMA model was implemented using an automated approach to select optimal parameters based on AIC (Akaike Information Criterion). The selected model was

ARIMA(1,1,1), which captures trend through differencing and models relationships using past values and error terms. The model did not include a seasonal component, indicating that seasonality was not strongly present in the data.

4. Data Preparation

The dataset *FinalWeeklyRidership* was used for this analysis.

The following preprocessing steps were performed:

- Passenger counts were aggregated at the weekly level across all carriers
- The *WeekStartDate* column was converted to datetime format
- The dataset was sorted and indexed by date
- Weekly frequency (W-MON) was enforced to maintain consistency
- Missing values were handled using interpolation

These steps ensured that the dataset was properly structured for time series modeling.

5. Model Implementation

The independent variable used in this model is time (*WeekStartDate*), and the dependent variable is *PassengerCount*. These variables are essential for capturing temporal patterns in passenger demand.

- Exponential Smoothing was applied using an additive trend component without seasonality
- ARIMA was implemented using auto_arima for automatic parameter selection
- Forecasts were generated for 260 weeks, covering the period from 2026 to 2030
- Visualization plots were created to compare actual and forecasted values

6. Results (Model Performance Evaluation)

Model performance was evaluated using two metrics:

- **Mean Absolute Error (MAE)**
- **Root Mean Squared Error (RMSE)**

Results:

- Exponential Smoothing MAE: **61,993.68**
- ARIMA MAE: **66,633.95**
- Exponential Smoothing RMSE: **62,435.62**
- ARIMA RMSE: **67,096.78**

Both MAE and RMSE consistently indicate that the Exponential Smoothing model performs better than the ARIMA model.

RMSE further confirms the result by penalizing larger errors, and the Exponential Smoothing model consistently shows lower error values.

RMSE penalizes larger errors more heavily than MAE, making it useful for identifying models with higher variance in predictions.

7. Interpretation

The forecast results show a steady increase in passenger demand over time. This indicates a recovery and growth trend in bus terminal usage.

Although minor downward fluctuations were observed at the end of each year, the seasonal effect was not strong enough to significantly improve model accuracy when tested using non-seasonal models.

The increasing trend suggests higher demand in the coming years, especially during peak periods. This information can support capacity planning, scheduling, and infrastructure development at the bus terminal.

This suggests that long-term trend plays a more significant role than seasonal variation in this dataset.

8. Conclusion

Based on model performance and error comparison, Exponential Smoothing (Holt's Linear Trend model) is selected as the final forecasting model. It provides more accurate predictions and effectively captures long-term trends in passenger demand.

This model can support planning and decision-making for future operations at the Port Authority Bus Terminal.

9. Attachments / Proof of Work

Figure 1: Exponential Smoothing Forecast

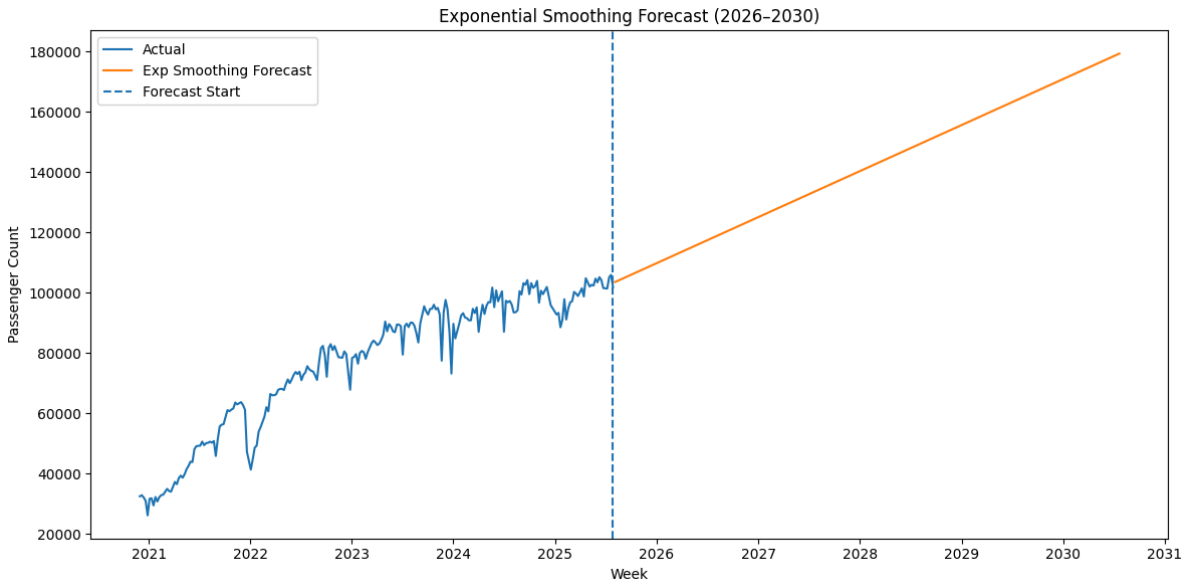

Figure 2: ARIMA Forecast

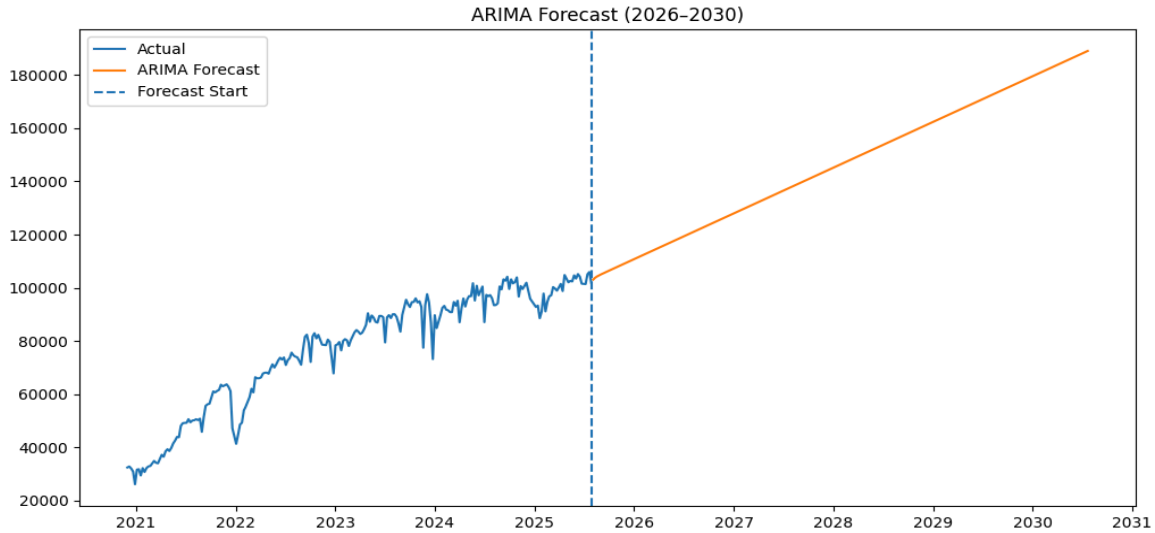

Figure 3: Model Output (Terminal Results)

```
(myenv) sherinsreedharan@sherins-MacBook-Air Downloads % nano forecast.py
(myenv) sherinsreedharan@sherins-MacBook-Air Downloads % nano forecast.py
(myenv) sherinsreedharan@sherins-MacBook-Air Downloads % python forecast.py
Performing stepwise search to minimize aic
 ARIMA(2,1,2)(0,0,0)[0] intercept   : AIC=inf, Time=0.26 sec
 ARIMA(0,1,0)(0,0,0)[0] intercept   : AIC=4691.380, Time=0.03 sec
 ARIMA(1,1,0)(0,0,0)[0] intercept   : AIC=4679.123, Time=0.05 sec
 ARIMA(0,1,1)(0,0,0)[0] intercept   : AIC=4674.819, Time=0.07 sec
 ARIMA(0,1,0)(0,0,0)[0]             : AIC=4690.798, Time=0.04 sec
 ARIMA(1,1,1)(0,0,0)[0] intercept   : AIC=4671.005, Time=0.18 sec
 ARIMA(2,1,1)(0,0,0)[0] intercept   : AIC=4676.101, Time=0.08 sec
 ARIMA(1,1,2)(0,0,0)[0] intercept   : AIC=4673.137, Time=0.10 sec
 ARIMA(0,1,2)(0,0,0)[0] intercept   : AIC=4671.902, Time=0.14 sec
 ARIMA(2,1,0)(0,0,0)[0] intercept   : AIC=4676.548, Time=0.05 sec
 ARIMA(1,1,1)(0,0,0)[0]             : AIC=4674.968, Time=0.06 sec

Best model:  ARIMA(1,1,1)(0,0,0)[0] intercept
Total fit time: 1.061 seconds
ARIMA model built successfully
Exponential Smoothing MAE: 61993.67709356695
ARIMA MAE: 66633.94999740958
```

```
(myenv) sherinsreedharan@sherins-MacBook-Air Downloads % nano forecast.py
(myenv) sherinsreedharan@sherins-MacBook-Air Downloads % nano forecast.py
(myenv) sherinsreedharan@sherins-MacBook-Air Downloads % nano forecast.py
(myenv) sherinsreedharan@sherins-MacBook-Air Downloads % nano forecast.py
(myenv) sherinsreedharan@sherins-MacBook-Air Downloads % python forecast.py
Performing stepwise search to minimize aic
 ARIMA(2,1,2)(0,0,0)[0] intercept   : AIC=inf, Time=0.31 sec
 ARIMA(0,1,0)(0,0,0)[0] intercept   : AIC=4691.380, Time=0.02 sec
 ARIMA(1,1,0)(0,0,0)[0] intercept   : AIC=4679.123, Time=0.04 sec
 ARIMA(0,1,1)(0,0,0)[0] intercept   : AIC=4674.819, Time=0.05 sec
 ARIMA(0,1,0)(0,0,0)[0]             : AIC=4690.798, Time=0.03 sec
 ARIMA(1,1,1)(0,0,0)[0] intercept   : AIC=4671.005, Time=0.15 sec
 ARIMA(2,1,1)(0,0,0)[0] intercept   : AIC=4676.101, Time=0.08 sec
 ARIMA(1,1,2)(0,0,0)[0] intercept   : AIC=4673.137, Time=0.10 sec
 ARIMA(0,1,2)(0,0,0)[0] intercept   : AIC=4671.902, Time=0.14 sec
 ARIMA(2,1,0)(0,0,0)[0] intercept   : AIC=4676.548, Time=0.05 sec
 ARIMA(1,1,1)(0,0,0)[0]             : AIC=4674.968, Time=0.07 sec

Best model:  ARIMA(1,1,1)(0,0,0)[0] intercept
Total fit time: 1.059 seconds
ARIMA model built successfully
Exponential Smoothing MAE: 61993.67709356695
ARIMA MAE: 66633.94999740958
Exponential Smoothing RMSE: 62435.61741888972
ARIMA RMSE: 67096.78466996466
(myenv) sherinsreedharan@sherins-MacBook-Air Downloads %
```

Figure 4: Code Implementation

```
UW PICO 5.09                                                    File: forecast.py

import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from pmdarima import auto_arima

# Load dataset
df = pd.read_excel("FinalWeeklyRidership.xlsx")

# Convert date column
df["WeekStartDate"] = pd.to_datetime(df["WeekStartDate"])

# Aggregate weekly passenger count
weekly_df = df.groupby("WeekStartDate")["PassengerCount"].sum().reset_index()

# Sort and set index
weekly_df = weekly_df.sort_values("WeekStartDate")
weekly_df.set_index("WeekStartDate", inplace=True)

# Set weekly frequency
weekly_df = weekly_df.asfreq("W-MON")

# Fill missing values
weekly_df["PassengerCount"] = weekly_df["PassengerCount"].interpolate()

# ----------------------------
# Exponential Smoothing Model
# ----------------------------
exp_model = ExponentialSmoothing(
    weekly_df["PassengerCount"],
    trend="add",
    seasonal=None
)
exp_fit = exp_model.fit()

exp_forecast = exp_fit.forecast(steps=260)

# Future dates
future_dates = pd.date_range(
    start=weekly_df.index[-1] + pd.Timedelta(weeks=1),
    periods=260,
    freq="W-MON"
)

exp_forecast = pd.Series(exp_forecast.values, index=future_dates)

# Plot Exponential Smoothing
plt.figure(figsize=(12,6))
plt.plot(weekly_df.index, weekly_df["PassengerCount"], label="Actual")
```

UW PICO 5.09 File: forecast.py

```
#plt.show()

# ----------------------------
# ARIMA Model
# ----------------------------
# ARIMA Model
from pmdarima import auto_arima

arima_model = auto_arima(
    weekly_df["PassengerCount"],
    seasonal=False,
    trace=True,
    error_action="ignore",
    suppress_warnings=True
)

arima_forecast = arima_model.predict(n_periods=260)

future_dates_arima = pd.date_range(
    start=weekly_df.index[-1] + pd.Timedelta(weeks=1),
    periods=260,
    freq="W-MON"
)

arima_forecast = pd.Series(arima_forecast, index=future_dates_arima)

print("ARIMA model built successfully")

# Plot ARIMA
plt.figure(figsize=(12,6))
plt.plot(weekly_df.index, weekly_df["PassengerCount"], label="Actual")
plt.plot(arima_forecast.index, arima_forecast, label="ARIMA Forecast")
plt.axvline(x=weekly_df.index[-1], linestyle="--", label="Forecast Start")
plt.title("ARIMA Forecast (2026–2030)")
plt.legend()

plt.savefig("arima_forecast.png")

plt.show(block=False)
plt.close()

from sklearn.metrics import mean_absolute_error

# Align actual and predicted (last part)
actual = weekly_df["PassengerCount"]

exp_pred = exp_forecast[:len(actual)]
arima_pred = arima_forecast[:len(actual)]

# Calculate MAE
exp_mae = mean_absolute_error(actual, exp_pred)
arima_mae = mean_absolute_error(actual, arima_pred)

print("Exponential Smoothing MAE:", exp_mae)
print("ARIMA MAE:", arima_mae)
```